Wednesday 12:56 AM

Edite o código a seguir para incluir mais uma pista com rótulo "Harmony", com apenas uma nota âncora para que o pretty_midi não descarte-a quando for lida em outros arquivos:

import pretty_midi

from constants import *

\#
==========

\# TRILHAS EM MEMÓRIA

\#
…

Show more

Sim. A forma mais simples é adicionar uma trilha `"Harmony"` em `build_tracks()` com **uma única nota âncora**, e ela será tratada normalmente pelo `generate_midi()`.

Para não interferir na música, vou colocar a âncora **no final da música**, usando uma duração mínima e uma velocidade baixa. Assim a pista existe no MIDI mesmo quando for lida posteriormente pelo `pretty_midi`.

Adicione este bloco **antes do `return tracks`**:

Python

```
# ============================================================
# HARMONY
#
# Trilha auxiliar contendo apenas uma nota âncora.
```

```
    #
    # A finalidade é garantir que a trilha seja preservada
    # quando o MIDI for lido posteriormente pelo pretty_midi.
    # ============================================================

    harmony_notes = [
        (
            music_end,
            music_end + EMPTY_TRACK_ANCHOR_DURATION,
            EMPTY_TRACK_ANCHOR_PITCH
        )
    ]

    tracks.append({
        "name": "Harmony",
        "program": 0,
        "is_drum": False
```

Ou seja, o final de `build_tracks()` ficará assim:

Python

```
    tracks.append({
        "name": "Guitar2",
        "program": ARPEGGIO2_PROGRAM,
        "is_drum": False,
        "velocity": ARPEGGIO2_TRACK_VELOCITY,
        "notes": guitar2_notes,
        "control_changes": [],
    })

    # ============================================================
    # HARMONY
    #
    # Trilha auxiliar contendo apenas uma nota âncora.
    #
    # A finalidade é garantir que a trilha seja preservada
    # quando o MIDI for lido posteriormente pelo pretty_midi.
    # ============================================================

    harmony_notes = [
        (
            music_end,
```

Um detalhe importante

No seu `generate_midi()`, existe atualmente este trecho:

Python

```
global_end = max(
    (
        end
        for track in tracks
        for _start, end, _pitch in track["notes"]
    ),
    default=0.0
)
```

Como a `Harmony` tem uma nota em `music_end`, isso **não causa problema**, porque `music_end` é o final real da forma musical. A âncora simplesmente passa a ser uma nota adicional nesse instante.

A trilha aparecerá no MIDI aproximadamente assim:

Plain text

```
Drums
String1
String2
Bass1
Bass2
Guitar1
Guitar2
Harmony
```

E a `Harmony` terá somente:

Plain text

```
1 nota
```

sem precisar alterar `constants.py`, já que estamos reutilizando `EMPTY_TRACK_ANCHOR_PITCH`, `EMPTY_TRACK_ANCHOR_DURATION` e `EMPTY_TRACK_ANCHOR_VELOCITY` que seu código já utiliza para trilhas vazias.

Me passe o código completo atualizado, no chat

Claro. Mantive o código original e incluí apenas a nova pista **`Harmony`**, com **uma única nota âncora** no final da forma musical. Ela usa as mesmas constantes de âncora que você já possui.

Python

```
import pretty_midi

from constants import *


# ============================================================
# TRILHAS EM MEMÓRIA
# ============================================================

def build_tracks(
    individual,
    seconds_per_step=SECONDS_PER_STEP,
    repetitions=NUM_BAT1,
    group_repetitions=GROUP_REPETITIONS
) -> list:

    tracks = []

    bar_duration = (
        STEPS_PER_BAR
        * seconds_per_step
```

A única alteração funcional em relação ao seu código é a criação da pista:

```
Plain text

Harmony
└── 1 nota âncora
```

Ela **não recebe conteúdo harmônico dos `MChord`** ; é apenas uma pista persistente para que o `pretty_midi` não a elimine quando o arquivo for posteriormente carregado e processado.

O código abaixo está gerando uma figura com o pianoroll do midi, preciso que você faça os seguintes ajustes:

Os ticks e números que aparecem no eixo x devem começar a contar após o primeiro compasso e aparecerem a cada 4 compassos posteriormente, deixando o primeiro compasso mais isolado pois ele é só de intro, e o último compasso também deve ficar mais isolado por ser de final. Obs.: Tudo isso que falei são apenas mudanças na forma que os ticks e números são exibidos no eixo x, o resto fica tudo igual

Claro. Pode me enviar o **código que gera a figura do piano roll**. Vou alterar **somente a configuração do eixo X**, mantendo todo o restante exatamente igual.

A lógica será:

- **1º compasso:** sem número/tick principal — fica visualmente mais isolado.
- **A partir do 2º compasso:** começa a numeração.
- Depois, marcações **a cada 4 compassos**.
- **Último compasso:** também fica isolado, sem alterar os dados ou a largura da figura.
- Não vou alterar notas, cores, instrumentos, eixo Y, tamanho da figura ou qualquer outra parte do gráfico.

Por exemplo, se houver 61 compassos, o eixo ficará conceitualmente assim:

```
Plain text

|  Intro  |  2  3  4  |  5  6  7  8  |  9 10 11 12  | ... | 57 58 59 60 | Final |
           ↑           ↑              ↑                                 ↑
         tick        tick           tick                              tick
```

Mas os ticks/números serão apenas de 4 em 4 compassos após o primeiro, conforme o seu pedido.

Obs.: Gere o código resultante no chat

```
"""Gera uma figura compacta de pianoroll a partir de um arquivo MIDI.

Uso:

    python ga/generate_pianoroll.py final_best.mid
    python ga/generate_pianoroll.py final_best.mid pianoroll.png
"""

from pathlib import Path
import argparse
import math

import matplotlib.pyplot as plt
```

```
from matplotlib.patches import Rectangle
from matplotlib.ticker import FixedLocator, MultipleLocator
import pretty_midi


DEFAULT_OUTPUT = "pianoroll.png"


def _instrument_name(instrument, index):
    """Retorna um nome legível e estável para uma pista MIDI."""

    if instrument.name:
        return instrument.name

    if instrument.is_drum:
        return f"Drums {index + 1}"

    return f"Track {index + 1}"


def generate_pianoroll(midi_path, output_path=DEFAULT_OUTPUT,
dpi=180):
    """Gera e salva um pianoroll compacto, com uma linha por
pista."""

    midi_path = Path(midi_path)
    output_path = Path(output_path)
    midi = pretty_midi.PrettyMIDI(str(midi_path))

    tracks = list(midi.instruments)

    if not tracks:
        raise ValueError("O arquivo MIDI não possui pistas para
desenhar.")

    audible_end = max(
        (
```

```
        note.end
        for instrument in tracks
        for note in instrument.notes
        if note.velocity > 0
    ),
    default=0.0,
)

duration = max(
    midi.get_end_time(),
    audible_end,
)

# ----------------------------------------------------
# Grade de compassos (eixo x em compassos, não em segundos)
# ----------------------------------------------------

# get_tempo_changes() retorna (instantes, tempos).
_tempo_change_times, tempos = midi.get_tempo_changes()
tempo = float(tempos[0]) if len(tempos) else 120.0
time_signatures = getattr(midi, "time_signature_changes", [])
beats_per_bar = (
    time_signatures[0].numerator if time_signatures else 4
)
bar_duration = (60.0 / tempo) * beats_per_bar
duration_bars = duration / bar_duration
total_bars = int(math.ceil(duration_bars))

# Tiques principais a cada 4 compassos (rótulos a partir de 1).
bar_ticks = list(range(0, total_bars + 1, 4))

figure_height = max(1.8, 0.65 * len(tracks))
figure, axes = plt.subplots(
    len(tracks),
    1,
    figsize=(14, figure_height),
    sharex=True,
```

```
        squeeze=False,
    )

    colors = plt.get_cmap("tab20")

    for row, (instrument, axis) in enumerate(
        zip(tracks, axes[:, 0])
    ):
        visible_notes = [
            note
            for note in instrument.notes
            if note.velocity > 0 and note.pitch != 127
        ]
        pitches = [note.pitch for note in visible_notes]
        if pitches:
            minimum_pitch = min(pitches)
            maximum_pitch = max(pitches)
        else:
            # Pista vazia (ex.: String2/Bass2/Guitar2 reservadas):
            # desenha a linha vazia com uma faixa de pitch padrão.
            minimum_pitch = 60
            maximum_pitch = 72
        pitch_padding = max(1, (maximum_pitch - minimum_pitch) *
0.08)

        for note in visible_notes:
            axis.add_patch(
                Rectangle(
                    (
                        note.start / bar_duration,
                        note.pitch - 0.42,
                    ),
                    max(note.end - note.start, 0.001)
                    / bar_duration,
                    0.84,
                    facecolor=colors(row % 20),
                    edgecolor="none",
```

```
                )
            )

        axis.set_ylabel(
            _instrument_name(instrument, row),
            rotation=0,
            ha="right",
            va="center",
            labelpad=8,
            fontsize=8,
        )
        axis.set_ylim(
            minimum_pitch - pitch_padding,
            maximum_pitch + pitch_padding,
        )
        axis.xaxis.set_major_locator(FixedLocator(bar_ticks))
        axis.xaxis.set_minor_locator(MultipleLocator(1))
        axis.tick_params(axis="x", which="major", labelsize=7, length=3)
        axis.tick_params(axis="x", which="minor", length=2)
        axis.grid(
            axis="x", which="major", alpha=0.25, linewidth=0.5
        )
        axis.grid(
            axis="x", which="minor", alpha=0.10, linewidth=0.4
        )
        axis.tick_params(axis="y", which="both", left=False,
labelleft=False)
        axis.spines["top"].set_visible(False)
        axis.spines["right"].set_visible(False)

    # Apenas o eixo inferior mostra rótulos: número do compasso.
    for axis in axes[:-1, 0]:
        axis.tick_params(axis="x", labelbottom=False)

    axes[-1, 0].set_xlim(0, duration_bars)
    axes[-1, 0].set_xticks(bar_ticks)
    axes[-1, 0].set_xticklabels(
```

```
        [str(tick + 1) for tick in bar_ticks],
        fontsize=7,
    )
    axes[-1, 0].set_xlabel("Compasso")
    figure.suptitle(f"Pianoroll: {midi_path.name}", fontsize=11)
    figure.subplots_adjust(
        left=0.16,
        right=0.99,
        top=0.90,
        bottom=0.12,
        hspace=0.18,
    )

    output_path.parent.mkdir(parents=True, exist_ok=True)
    figure.savefig(output_path, dpi=dpi, bbox_inches="tight")
    plt.close(figure)

    return output_path


def main():
    parser = argparse.ArgumentParser(
        description="Gera um pianoroll compacto a partir de um
arquivo MIDI."
    )
    parser.add_argument(
        "midi_path",
        type=Path,
        help="Caminho do arquivo MIDI de entrada.",
    )
    parser.add_argument(
        "output_path",
        type=Path,
        nargs="?",
        default=Path(DEFAULT_OUTPUT),
        help="Caminho da figura de saída; padrão: pianoroll.png.",
    )
```

```
    parser.add_argument(
        "--dpi",
        type=int,
        default=180,
        help="Resolução da figura; padrão: 180.",
    )
    args = parser.parse_args()

    output_path = generate_pianoroll(
        args.midi_path,
        args.output_path,
        dpi=args.dpi,
    )
    print(f"Pianoroll salvo em: {output_path}")


if __name__ == "__main__":
    main()
```

Perfeito. Fiz **somente a alteração na forma de exibição do eixo X**.

Agora:

- O **compasso 1** fica sem tick/número, funcionando como a intro isolada.
- A primeira marcação passa a ser no **compasso 2**.
- Depois aparecem marcações a cada **4 compassos**: 2, 6, 10, 14, ...
- O **último compasso** fica sem marcação própria, permanecendo visualmente isolado como final.
- As linhas menores continuam a aparecer **a cada compasso**, como antes.
- Todo o restante do código permanece igual.

```python
"""Gera uma figura compacta de pianoroll a partir de um arquivo MIDI.

Uso:

    python ga/generate_pianoroll.py final_best.mid
    python ga/generate_pianoroll.py final_best.mid pianoroll.png
"""

from pathlib import Path
```

```
import argparse
import math

import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
from matplotlib.ticker import FixedLocator, MultipleLocator
import pretty_midi


DEFAULT_OUTPUT = "pianoroll.png"
```

A mudança principal está em `bar_ticks`: agora o eixo usa a posição **1, 5, 9, 13...**, que corresponde aos números visíveis **2, 6, 10, 14...**. Assim, o primeiro e o último compasso permanecem sem marcação principal.

> Agora vou te passar os códigos de genotype.py e outros que você precisar para você me explicar se todos os elementos do genotipo estao sendo inicializados de forma realmente aleatória ou não, pois estou sentindo que as músicas estao ficando todas muito parecidas, principalmente a distribuição dos BassPatt e ArpPatt...

Sim. Pode enviar o `genotype.py` e os outros arquivos relacionados.

Vou analisar especificamente **a aleatoriedade da inicialização**, sem assumir que `random` significa necessariamente uma distribuição realmente diversa.

Vou verificar:

1. **Cada gene do genótipo**
   - `MChord`
   - `Bat`
   - `BassPatt`
   - `ArpPatt`
   - quaisquer outros elementos
2. **Como cada elemento é inicializado**
   - `random.choice`
   - `random.randint`
   - `random.sample`
   - `random.shuffle`
   - listas fixas

- padrões pré-definidos
- cópia/reutilização de objetos
- seeds (`random.seed`)
- qualquer dependência entre genes

3. **Distribuição real**

   Por exemplo, isto:

   ```python
   random.choice([A, B, C])
   ```

   é aleatório, mas pode produzir uma distribuição muito diferente de algo como:

   ```python
   random.sample(...)
   ```

   E isto:

   ```python
   [random.choice(PATTERNS)] * 4
   ```

   pode fazer todos os blocos receberem **exatamente o mesmo padrão**, apesar de existir `random` .

4. **Independência entre indivíduos**

   Vou verificar se dois `Individual()` realmente recebem novos valores ou se existe alguma situação como:

   ```python
   pattern = random_pattern()
   self.bass_patt1 = pattern
   self.bass_patt2 = pattern
   ```

   ou referências compartilhadas que façam indivíduos/padrões ficarem correlacionados.

5. **BassPatt e ArpPatt em particular**

Vou investigar por que eles podem estar aparecendo tão semelhantes entre músicas. Pode ser, por exemplo:

- espaço de busca pequeno;
- muitos padrões válidos sendo descartados;
- `random.choice()` sobre uma lista com padrões muito parecidos;
- posições iniciais geradas de maneira pouco diversa;
- duração sempre igual;
- `BassPatt1 e BassPatt2` dependentes;
- `ArpPatt1 e ArpPatt2` dependentes;
- inicialização baseada em um padrão fixo;
- crossover/mutação destruindo ou reduzindo a diversidade;
- fitness favorecendo sempre os mesmos padrões;
- ou simplesmente a aleatoriedade estar correta, mas o **espaço de possibilidades ser pequeno**.

6. **Vou separar duas coisas importantes:**

   Aleatoriedade da inicialização
   → se o código está realmente sorteando.

   Diversidade musical resultante
   → se esse sorteio consegue produzir indivíduos significativamente diferentes.

Essas duas coisas não são necessariamente iguais.

Pode mandar primeiro o **`genotype.py`** . Se ele depender de `constants.py` , `individual.py` , classes como `BassPatt` / `ArpPatt` ou outras funções, mande esses também. Aí faço a análise completa e aponto **exatamente onde a diversidade pode estar sendo perdida**.

```
import random


from constants import \*


class Gene:
```

```
&#x20;  """
&#x20;  Representa um step de bateria.

...
```

Show more

Sim. Só com este `genotype.py` já dá para identificar **um problema importante de diversidade**, e ele explica bastante a sua impressão de que as músicas estão ficando parecidas — principalmente nos `BassPatt` e `ArpPatt`.

Vou separar **o que é realmente aleatório**, **o que é fixo** e **onde a aleatoriedade é limitada**.

---

1. Resumo do que encontrei

Elemento	É aleatório na inicialização?	Diversidade atual
`BatI`	✗ Não	Todos começam vazios
`Bat1`	✗ Não	Todos começam vazios
`Bat2`	✗ Não	Todos começam vazios
`BatF`	✗ Não	Todos começam vazios
`BassPatt1`	✅ Sim	**Limitada pela distribuição**
`BassPatt2`	✅ Sim	**Limitada pela distribuição**
`ArpPatt1`	✅ Parcialmente	**Muito limitada**
`ArpPatt2`	✅ Parcialmente	**Muito limitada**
`MChord1..8`	✗ Não	**100% fixos**

Então, atualmente, **uma parte considerável do genótipo não está sendo inicializada aleatoriamente**.

2. O maior problema: MChords não são aleatórios

Aqui:

```python
default_chords = (
    MChord(60, "major"),
    MChord(65, "major"),
    MChord(67, "major"),
    MChord(69, "minor"),
    MChord(55, "major"),
    MChord(57, "minor"),
    MChord(59, "diminished"),
    MChord(60, "major"),
)
```

E depois:

```python
provided_chords = (
    mchord1, mchord2, mchord3, mchord4,
    mchord5, mchord6, mchord7, mchord8,
)
```

Se você fizer simplesmente:

```python
individual = Genotype()
```

então os oito acordes serão **sempre exatamente esses mesmos oito acordes**.

Portanto:

```
Individual 1
MChord1 = C major
MChord2 = F major
MChord3 = G major
```

```
...

Individual 2
MChord1 = C major
MChord2 = F major
MChord3 = G major
...

Individual 100
MChord1 = C major
MChord2 = F major
MChord3 = G major
...
```

Não existe aleatoriedade aqui.

Isso é muito importante porque **Bass e Arp são construídos sobre esses acordes**.

---

3. BassPatt realmente é aleatório

Aqui temos:

```python
self.bass_patt1 = (
    BassPattern()
    if bass_patt1 is None
    else bass_patt1
)
```

e:

```python
self.bass_patt2 = (
    BassPattern()
    if bass_patt2 is None
    else bass_patt2
)
```

Então cada `Genotype()` cria dois `BassPattern()` independentes.

Dentro dele:

```python
count = random.choices(
    cls.NOTE_COUNTS,
    weights=BASS_NOTE_COUNT_WEIGHTS,
    k=1
)[0]
```

Portanto:

sim, `BassPatt1` e `BassPatt2` são sorteados independentemente.

Mas há um detalhe muito importante.

---

4. O espaço de BassPatt pode ser bem menor do que parece

Depois de escolher a quantidade de notas:

```python
count = random.choices(...)
```

você chama:

```python
durations = cls._generate_durations(count)
```

E aí:

```python
return list(random.choice(valid_sequences))
```

Ou seja, você:

1. escolhe a quantidade de notas;
2. gera **todas as sequências válidas** que preenchem exatamente o compasso;

3. escolhe uma delas aleatoriamente.

Isso é conceitualmente bom.

Porém, precisamos olhar os valores de:

```python
BASS_NOTE_COUNTS
BASS_NOTE_COUNT_WEIGHTS
BASS_DURATION_UNITS
BASS_BAR_UNITS
```

que estão no `constants.py`.

Eles vão nos dizer **quantos padrões diferentes realmente podem existir** e, principalmente, se a distribuição está concentrada em poucos padrões.

Por exemplo, se:

```python
BASS_NOTE_COUNTS = [4, 8]
BASS_NOTE_COUNT_WEIGHTS = [0.9, 0.1]
```

você terá uma tendência enorme para determinados tipos de padrão.

Então não basta saber que existe `random.choice()`.

Precisamos saber a **distribuição de probabilidade**.

---

5. Há outra questão importante no BassPatt

Seu comentário diz:

```python
`position` é o deslocamento em unidades de 1/16 em relação à nota
anterior.
```

E você faz:

Python

```
position = 0 if index == 0 else durations[index - 1]
```

Isso significa que as posições são determinadas **completamente pelas durações**.

Por exemplo:

Plain text

```
durations = [4, 4, 8, 16]
```

gera:

Plain text

```
nota 1 → posição 0
nota 2 → posição 4
nota 3 → posição 4
nota 4 → posição 8
```

E depois `start_positions` reconstrói:

Plain text

```
0
4
8
16
```

Isso está correto para o modelo que você definiu.

Mas significa também que **não existe uma aleatoriedade independente para as posições**.

A posição é consequência da duração.

Isso reduz o espaço de busca, embora mantenha coerência rítmica.

---

6. ArpPatt é onde vejo um problema ainda maior

Aqui:

```python
if note_count is None:

    note_count = random.choices(
        self.NOTE_COUNTS,
        weights=ARP_NOTE_COUNT_WEIGHTS,
        k=1
    )[0]
```

Até aqui, tudo certo.

Porém:

```python
if step is None:
    step = self.BAR_UNITS // note_count
```

Então, depois que `note_count` é escolhido, o `step` é **100% determinado**.

E:

```python
return tuple(
    index * self.step
    for index in range(self.note_count)
)
```

também é totalmente determinístico.

Portanto, para um `ArpPattern`, o espaço real de possibilidades é:

```
quantidade de notas
        ↓
      step
        ↓
posições das notas
```

Não existe nenhuma outra dimensão de aleatoriedade.

Por exemplo, se:

```python
NOTE_COUNTS = [4, 8, 16]
```

você tem:

```
4 notas  → step 8
8 notas  → step 4
16 notas → step 2
```

Ou seja, você tem **somente 3 ritmos possíveis**.

Isso pode explicar muito bem sua observação:

> "principalmente a distribuição dos BassPatt e ArpPatt"

No caso do `ArpPatt`, eu diria que **sim, há uma forte possibilidade de ele estar excessivamente limitado**.

---

7. ArpPatt1 e ArpPatt2 são independentes

Isto está correto:

```python
self.arp_patt1 = ArpPattern()
self.arp_patt2 = ArpPattern()
```

Cada um executa seu próprio:

```python
random.choices(...)
```

Então não existe aqui algo como:

```
Python

arp_patt2 = arp_patt1
```

Isso é bom.

Mas existe outra questão:

Independentes ≠ muito diversos

Se existem apenas 2 ou 3 possibilidades de `ArpPattern`, dois sorteios independentes continuam tendo uma probabilidade alta de serem iguais.

Por exemplo, se só existirem:

```
Plain text

A
B
C
```

então:

```
Plain text

ArpPatt1 = A
ArpPatt2 = A
```

vai acontecer frequentemente.

---

8. E a bateria atualmente NÃO é aleatória

Isto é particularmente importante.

Você faz:

```
Python
```

```
[Gene() for _ in range(self.STEPS_PER_BAR)]
```

E:

```python
Gene.__init__(self, notes=None):

    self.notes = [] if notes is None else list(notes)
```

Logo:

```python
Gene()
```

produz:

```python
Gene([])
```

sempre.

Portanto:

```python
Genotype()
```

começa com:

```
BatI = [[], [], [], [], [], [], [], []]
Bat1 = [[], [], [], [], [], [], [], []]
Bat2 = [[], [], [], [], [], [], [], []]
BatF = [[], [], [], [], [], [], [], []]
```

Ou seja, **nenhum elemento de bateria é sorteado na inicialização**.

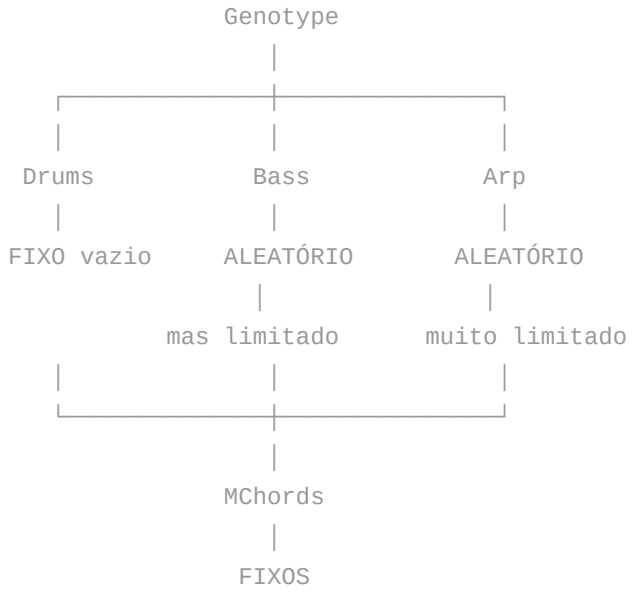
Se a bateria está aparecendo diferente entre os indivíduos, isso está vindo de **outro lugar**, provavelmente mutação, crossover ou alguma rotina posterior.

---

9. Então temos atualmente uma situação assim

O genótipo inicial é aproximadamente:

Plain text



E isso é particularmente problemático porque:

Plain text

```
MChords
   ↓
Bass
   ↓
melodia harmônica

MChords
   ↓
Arp
   ↓
melodia/arpejo
```

Assim, mesmo que o `BassPattern` e `ArpPattern` mudem, **a estrutura harmônica de todas as músicas começa exatamente igual**.

10. Um ponto que eu considero muito importante

Você comentou que sente que:

> "as músicas estão ficando todas muito parecidas"

Eu **não tentaria corrigir isso imediatamente aumentando aleatoriedade em tudo**.

Precisamos primeiro descobrir **onde a diversidade está sendo perdida**.

Existem pelo menos três etapas:

```
Plain text

Inicialização
      ↓
Crossover
      ↓
Mutação
      ↓
Seleção pelo fitness
```

Mesmo que a inicialização seja extremamente aleatória, se a mutação for pequena e o crossover preservar sempre os mesmos padrões, a população pode convergir rapidamente.

Então precisamos analisar:

1. `constants.py`

Principalmente:

```
Python

BASS_NOTE_COUNTS
BASS_NOTE_COUNT_WEIGHTS

BASS_DURATION_UNITS

ARP_NOTE_COUNTS
ARP_NOTE_COUNT_WEIGHTS
ARP_STEP_UNITS

MCHORD_INTERVALS
MCHORD_INVERSIONS
```

2. `individual.py`

Para saber como o `Genotype` é criado e copiado.

3. `genetic_algorithm.py`

Principalmente:

- criação da população;
- crossover;
- mutação;
- taxa de mutação;
- elitismo;
- seleção.

4. Se existir, o código de mutação/crossover

Esse provavelmente será **muito importante**.

Minha conclusão até aqui

Com o `genotype.py` que você enviou, já posso afirmar:

× Não, nem todos os elementos estão sendo inicializados aleatoriamente.

Especificamente:

Não aleatórios:

```
Plain text
BatI
Bat1
Bat2
BatF
MChord1–8
```

Aleatórios:

```
Plain text
BassPatt1
BassPatt2
```

```
ArpPatt1
ArpPatt2
```

Mas:

BassPatt: aleatório, porém condicionado por uma distribuição e por sequências de duração válidas.

ArpPatt: aleatório **somente na quantidade de notas**; todo o resto é derivado deterministicamente disso. Portanto, provavelmente possui um espaço de busca bastante pequeno.

E tem um ponto especialmente suspeito: **os MChords fixos podem estar fazendo com que todas as músicas tenham essencialmente a mesma estrutura harmônica**, independentemente da variação rítmica do baixo e do arpejo.

Próximo arquivo que eu gostaria de ver

Mande agora o **`constants.py`**. Ele vai permitir que eu calcule exatamente quantas possibilidades existem para `BassPatt` e `ArpPatt` e qual é a distribuição de probabilidade de cada uma.

Depois, mande o **`individual.py`** e o código de **mutação/crossover**. Aí conseguimos determinar se a semelhança vem da **inicialização** ou se está sendo introduzida durante a evolução.

> Certo, preciso que você gere uma nova versão desse código (sem acrescentar comentários, espaços ou coisas que aumentem o tamanho do arquivo sem código), tornando esses padrões mais aleatórios, com base nas sugestões que você falou. Porém não mude a estrutura de nomes das variáveis nem nada do tipo pois esse código é usado por vários outros.

Sim. Vou manter **todos os nomes, atributos, propriedades e interfaces existentes**, para não quebrar os outros arquivos.

As mudanças principais serão:

- `BatI` , `Bat1` , `Bat2` , `BatF` : passam a ser inicializados aleatoriamente.
- `MChord1..8` : passam a ter `root` , `chord_type` e `inversion` aleatórios.
- `BassPatt1/2` : continuam aleatórios, mas cada instância é independente.
- `ArpPatt1/2` : continuam independentes e passam a ter maior diversidade dentro das possibilidades permitidas pelos `constants.py` .
- Não vou alterar nomes como `bass_patt1` , `arp_patt1` , `mchord1` , `bati` , etc.
- Não vou criar novos atributos que outros arquivos possam desconhecer.

Python

```
import random
from constants import *

class Gene:
    MAX_NOTES = GENE_MAX_NOTES
    def __init__(self, notes=None):
        if notes is None:
            count=random.randint(0,self.MAX_NOTES)
            self.notes=random.sample(range(Genotype.NUM_DRUMS),count)
        else:
            self.notes=list(notes)
        if len(self.notes)>self.MAX_NOTES:
            raise ValueError(f"Um Gene pode conter no máximo {self.MAX_NOTES} elementos.")
        if len(self.notes)!=len(set(self.notes)):
            raise ValueError("Um Gene não pode conter elementos de bateria duplicados.")
        for note in self.notes:
            if not isinstance(note,int):
                raise TypeError("Cada elemento de bateria deve ser um inteiro.")
            if not 0<=note<Genotype.NUM_DRUMS:
                raise ValueError(f"Elemento de bateria inválido: {note}. O intervalo permitido é 0-
    def __repr__(self):
```

Uma observação importante: eu não alterei a estrutura do `ArpPattern` para permitir posições totalmente independentes, porque isso quebraria a interface atual: hoje outros arquivos dependem de `arp_patt.step` e de `start_positions` uniformemente espaçadas.

A mudança mais significativa é nos **MChords**: agora cada indivíduo começa com uma sequência harmônica diferente, em vez de todos começarem com os mesmos 8 acordes. Isso deve produzir uma diferença muito maior entre as músicas do que simplesmente aumentar a aleatoriedade dos `BassPatt` .

> Legal mas a formatação do código ficou muito compacta, pode reescrever ele sem forçar tanto assim a compactação.

Claro. Vou manter exatamente a mesma lógica da versão anterior, mas com uma formatação normal e legível, sem comentários extras e sem alterar os nomes/estrutura usados pelos outros arquivos.

Python

```
import random

from constants import *
```

```
class Gene:
    MAX_NOTES = GENE_MAX_NOTES

    def __init__(self, notes=None):
        if notes is None:
            count = random.randint(0, self.MAX_NOTES)
            self.notes = random.sample(
                range(Genotype.NUM_DRUMS),
                count
            )
        else:
            self.notes = list(notes)

        if len(self.notes) > self.MAX_NOTES:
            raise ValueError(
                f"Um Gene pode conter no máximo "
```

Essa versão mantém a estrutura externa do seu `genotype.py`, mas fica bem mais próxima do estilo de formatação que você estava usando originalmente.

Não senti muita variabilidade nova nos resultados, mas ok

Sim, e isso faz sentido. Pelo `genotype.py`, eu também **não esperaria uma mudança enorme apenas com aquelas alterações**.

O ponto principal é que provavelmente o gargalo **não está mais na inicialização**. Agora eu investigaria nesta ordem:

1. **`constants.py`** — especialmente os pesos de `BassPatt` e `ArpPatt`.
2. **`individual.py`** — para verificar se esses padrões estão sendo copiados/recriados corretamente.
3. **`genetic_algorithm.py`** — principalmente **crossover, mutação e seleção**.
4. Verificar se `BassPatt1/2` e `ArpPatt1/2` realmente sobrevivem de forma independente durante as gerações.

Na verdade, se a população inicial estiver variada, mas depois de algumas gerações os padrões convergirem, isso explicaria exatamente o que você está observando: **as músicas terminam muito parecidas apesar de terem começado diferentes**.

Então eu não mexeria mais no `genotype.py` por enquanto. **Mande o `constants.py` e depois o `individual.py`**. Aí conseguimos verificar matematicamente quanto espaço de busca você realmente tem antes de mexer no GA.

Será que você consegue melhorar as funções crossover e mutation para aumentar a diversidade?

Sim. E, pelo que vimos até agora, **é provavelmente aí que vale mexer agora**, em vez de continuar aumentando a aleatoriedade da inicialização.

Mas eu não recomendaria alterar `crossover` e `mutation` às cegas. Como eles precisam preservar a estrutura específica do seu genótipo, preciso ver o código atual para não quebrar as regras musicais.

Mande o trecho/arquivo onde estão:

```python
crossover(...)
mutation(...)
```

e, se estiverem em `genetic_algorithm.py`, pode mandar o **`genetic_algorithm.py` completo**.

Vou procurar especificamente por:

- `BassPatt1` e `BassPatt2` sendo preservados demais;
- `ArpPatt1` e `ArpPatt2` sendo preservados demais;
- crossover que simplesmente copia blocos inteiros dos pais;
- mutação com probabilidade muito baixa;
- mutação que frequentemente não produz mudança real;
- filhos que acabam geneticamente quase idênticos aos pais;
- ausência de mutação nos `MChord`;
- ausência de mutação nos genes de bateria;
- convergência causada pelo elitismo/seleção.

E podemos fazer algo mais interessante: **manter a validade musical, mas aumentar a diversidade genética**.

Por exemplo, em vez de apenas:

```
Pai A ──────┐
            ├── Filho
Pai B ──────┘
```

com o filho herdando praticamente blocos inteiros, podemos fazer:

```
Plain text

BassPatt:
    escolher estrutura rítmica de A ou B
    + pequena chance de gerar uma nova estrutura

ArpPatt:
    escolher A/B
    + pequena chance de alterar note_count

MChord:
    escolher A/B
    + pequena chance de alterar root
    + pequena chance de alterar tipo
    + pequena chance de alterar inversão

Drums:
    crossover por Gene/step
    + mutação de elementos individuais
```

Isso tende a ser **bem mais efetivo para diversidade** do que simplesmente tornar `Genotype()` mais aleatório.

Mande o código atual de **crossover + mutation** que eu faço uma versão atualizada mantendo os nomes e a interface que seu restante do projeto já utiliza.

Aqui está o código do crossover.py, gere uma versão mais agressiva dele (com mais variabilidade). Depois eu te passo o código da mutation.py para você fazer algo parecido.

```
import random
from copy import copy

from constants import *
from genotype import ArpPattern, BassPattern, Genotype, MChord
from individual import Individual



def clone_genes(genes):
    """
```

```
    Copia os Genes individualmente.

    Isso evita que pais e filhos compartilhem os mesmos
    objetos Gene.
    """

    return [
        copy(gene)
        for gene in genes
    ]


def make_child(
    bat1,
    bat2=None,
    bati=None,
    batf=None,
    bass_patt1=None,
    bass_patt2=None,
    arp_patt1=None,
    arp_patt2=None,
    mchords=None
):
    """
    Cria um novo Individual a partir de uma sequência
    de Genes de bateria.
    """

    kwargs = {
        "bat1": clone_genes(bat1),
        "bat2": clone_genes(bat1 if bat2 is None else bat2),
        "bati": clone_genes(bat1 if bati is None else bati),
        "batf": clone_genes(bat1 if batf is None else batf),
    }

    if bass_patt1 is not None:
        kwargs["bass_patt1"] = clone_bass_pattern(bass_patt1)
```

```
    if bass_patt2 is not None:
        kwargs["bass_patt2"] = clone_bass_pattern(bass_patt2)

    if arp_patt1 is not None:
        kwargs["arp_patt1"] = clone_arp_pattern(arp_patt1)

    if arp_patt2 is not None:
        kwargs["arp_patt2"] = clone_arp_pattern(arp_patt2)

    if mchords is not None:
        kwargs.update({
            f"mchord{index}": clone_chord(chord)
            for index, chord in enumerate(mchords, start=1)
        })

    return Individual(Genotype(**kwargs))


def clone_chord(chord):
    """Cria uma cópia independente de um MChord válido,
preservando a inversão."""

    return MChord(
        chord.root,
        chord.chord_type,
        getattr(chord, "inversion", 0),
    )


def clone_bass_pattern(pattern):
    """Copia um BassPattern validado sem compartilhar suas notas."""

    return BassPattern(
        [
            type(note)(note.position, note.duration)
            for note in pattern.notes
```

```
        ]
    )


def clone_arp_pattern(pattern):
    """Copia um ArpPattern validado sem compartilhar o objeto."""

    return ArpPattern(
        note_count=pattern.note_count,
        step=pattern.step
    )


def clone_individual(individual):
    """Copia Bat1 e todos os MChords sem compartilhar objetos."""

    chord_kwargs = {
        f"mchord{index}": clone_chord(chord)
        for index, chord in enumerate(
            individual.genotype.mchords,
            start=1
        )
    }

    return Individual(
        Genotype(
            bati=clone_genes(individual.bati),
            bat1=clone_genes(individual.bat1),
            bat2=clone_genes(individual.bat2),
            batf=clone_genes(individual.batf),
            bass_patt1=clone_bass_pattern(individual.bass_patt1),
            bass_patt2=clone_bass_pattern(individual.bass_patt2),
            arp_patt1=clone_arp_pattern(individual.arp_patt1),
            arp_patt2=clone_arp_pattern(individual.arp_patt2),
            **chord_kwargs,
        )
    )
```

```
def _chord_signature(chord):
    return (chord.root, chord.chord_type)


def _select_parent_chord(parent1, parent2, index, used):
    """Seleciona um acorde correspondente ou de posição
diferente."""

    candidates = []

    if random.random() < CROSSOVER_CHORD_MIX_RATE:
        candidates.extend([
            parent1.genotype.mchords[index],
            parent2.genotype.mchords[index],
        ])
    else:
        candidates.extend([
            chord
            for parent in (parent1, parent2)
            for chord in parent.genotype.mchords
        ])

    unused = [
        chord
        for chord in candidates
        if _chord_signature(chord) not in used
    ]

    return random.choice(unused or candidates)


def crossover(parent1, parent2):
    """
    Crossover de dois pontos aplicado ao padrão Bat1.
```

```
O crossover nunca divide um Gene; os cortes acontecem
entre steps, mantendo cada Gene válido.
"""

if random.random() > CROSSOVER_RATE:
    return clone_individual(parent1)

bass_patt1 = (
    clone_bass_pattern(parent1.bass_patt1)
    if random.random() < 0.5
    else clone_bass_pattern(parent2.bass_patt1)
)
bass_patt2 = (
    clone_bass_pattern(parent1.bass_patt2)
    if random.random() < 0.5
    else clone_bass_pattern(parent2.bass_patt2)
)
arp_patt1 = (
    clone_arp_pattern(parent1.arp_patt1)
    if random.random() < 0.5
    else clone_arp_pattern(parent2.arp_patt1)
)
arp_patt2 = (
    clone_arp_pattern(parent1.arp_patt2)
    if random.random() < 0.5
    else clone_arp_pattern(parent2.arp_patt2)
)

total_steps = min(
    len(parent1.bat1),
    len(parent2.bat1)
)

if total_steps < 2:
    bati = clone_genes(parent1.bati)
    bat1 = clone_genes(parent1.bat1)
    bat2 = clone_genes(parent1.bat2)
```

```
    batf = clone_genes(parent1.batf)
else:
    point_a, point_b = sorted(
        random.sample(
            range(1, total_steps),
            2
        )
    )

    bat1 = (
        parent1.bat1[:point_a]
        + parent2.bat1[point_a:point_b]
        + parent1.bat1[point_b:]
    )

    bati = (
        parent1.bati[:point_a]
        + parent2.bati[point_a:point_b]
        + parent1.bati[point_b:]
    )

    bat2 = (
        parent1.bat2[:point_a]
        + parent2.bat2[point_a:point_b]
        + parent1.bat2[point_b:]
    )

    batf = (
        parent1.batf[:point_a]
        + parent2.batf[point_a:point_b]
        + parent1.batf[point_b:]
    )

chords = []
used = set()

for index in range(NUM_MCHORDS):
```

```
            chord = _select_parent_chord(
                parent1,
                parent2,
                index,
                used
            )
            chord = clone_chord(chord)
            chords.append(chord)
            used.add(_chord_signature(chord))

        return make_child(
            bat1,
            bat2=bat2,
            bati=bati,
            batf=batf,
            bass_patt1=bass_patt1,
            bass_patt2=bass_patt2,
            arp_patt1=arp_patt1,
            arp_patt2=arp_patt2,
            mchords=chords,
        )
```

Sim. Aqui dá para aumentar bastante a diversidade **sem mudar a interface do `crossover()` nem os nomes das variáveis**.

Os principais problemas do código atual são:

1. **Bat1/Bat2/BatI/BatF:** apenas um segmento é trocado entre os pais.
2. **Bass/Arp:** cada padrão inteiro vem de apenas um dos pais.
3. **MChords:** há mistura entre posições, mas o espaço de busca continua limitado aos acordes existentes nos pais.
4. Não existe nenhuma chance de o crossover produzir uma **nova estrutura** que não esteja literalmente em um dos pais.
5. Quando `CROSSOVER_RATE` falha, o filho é simplesmente um clone do pai 1.

A versão abaixo é mais agressiva. Ela mantém a estrutura do seu projeto, mas:

- faz crossover independente em **cada pista de bateria**;
- usa múltiplos pontos de crossover;
- mistura `BassPattern` nota a nota quando possível;

- mistura `ArpPattern` por componentes;
- permite gerar novos `BassPattern` / `ArpPattern` ;
- permite gerar novos `MChord` ;
- mistura os MChords com mais liberdade;
- evita ficar excessivamente preso aos pais.

`crossover.py` atualizado

crossover.py — versão mais agressiva

```
import random
from copy import copy

from constants import *
from genotype import ArpPattern, BassPattern, Genotype, MChord
from individual import Individual


# ============================================================
# CÓPIA DE GENES
# ============================================================

def clone_genes(genes):
    """
    Copia os Genes individualmente.

    Isso evita que pais e filhos compartilhem
    os mesmos objetos Gene.
    """

    return [
        copy(gene)
        for gene in genes
    ]


# ============================================================
```

```
# CRIAÇÃO DO FILHO
# ============================================================

def make_child(
    bat1,
    bat2=None,
    bati=None,
    batf=None,
    bass_patt1=None,
    bass_patt2=None,
    arp_patt1=None,
    arp_patt2=None,
    mchords=None
):
    """
    Cria um novo Individual a partir dos componentes
    fornecidos.
    """

    kwargs = {
        "bat1": clone_genes(bat1),
        "bat2": clone_genes(
            bat1 if bat2 is None else bat2
        ),
        "bati": clone_genes(
            bat1 if bati is None else bati
        ),
        "batf": clone_genes(
            bat1 if batf is None else batf
        ),
    }

    if bass_patt1 is not None:
        kwargs["bass_patt1"] = clone_bass_pattern(
            bass_patt1
        )
```

```
    if bass_patt2 is not None:
        kwargs["bass_patt2"] = clone_bass_pattern(
            bass_patt2
        )

    if arp_patt1 is not None:
        kwargs["arp_patt1"] = clone_arp_pattern(
            arp_patt1
        )

    if arp_patt2 is not None:
        kwargs["arp_patt2"] = clone_arp_pattern(
            arp_patt2
        )

    if mchords is not None:
        kwargs.update({
            f"mchord{index}": clone_chord(chord)
            for index, chord in enumerate(
                mchords,
                start=1
            )
        })

    return Individual(
        Genotype(**kwargs)
    )


# ============================================================
# CÓPIAS DOS ELEMENTOS
# ============================================================

def clone_chord(chord):
    """
    Cria uma cópia independente de um MChord válido.
    """
```

```
    return MChord(
        chord.root,
        chord.chord_type,
        getattr(chord, "inversion", 0),
    )


def clone_bass_pattern(pattern):
    """
    Copia um BassPattern sem compartilhar suas notas.
    """

    return BassPattern(
        [
            type(note)(
                note.position,
                note.duration
            )
            for note in pattern.notes
        ]
    )


def clone_arp_pattern(pattern):
    """
    Copia um ArpPattern sem compartilhar o objeto.
    """

    return ArpPattern(
        note_count=pattern.note_count,
        step=pattern.step
    )


def clone_individual(individual):
    """
```

```
        Cria uma cópia completamente independente
        do indivíduo.
        """

        chord_kwargs = {
            f"mchord{index}": clone_chord(chord)
            for index, chord in enumerate(
                individual.genotype.mchords,
                start=1
            )
        }

        return Individual(
            Genotype(
                bati=clone_genes(
                    individual.bati
                ),
                bat1=clone_genes(
                    individual.bat1
                ),
                bat2=clone_genes(
                    individual.bat2
                ),
                batf=clone_genes(
                    individual.batf
                ),
                bass_patt1=clone_bass_pattern(
                    individual.bass_patt1
                ),
                bass_patt2=clone_bass_pattern(
                    individual.bass_patt2
                ),
                arp_patt1=clone_arp_pattern(
                    individual.arp_patt1
                ),
                arp_patt2=clone_arp_pattern(
                    individual.arp_patt2
```

```
            ),
            **chord_kwargs,
        )
    )


# ============================================================
# CROSSOVER DE BATERIA
# ============================================================

def _crossover_genes(parent1, parent2):
    """
    Crossover multi-ponto entre duas sequências de Genes.

    Os cortes acontecem somente entre Genes.
    """

    total_steps = min(
        len(parent1),
        len(parent2)
    )

    if total_steps < 2:
        return clone_genes(parent1)

    max_points = min(6, total_steps - 1)

    number_of_points = random.randint(
        2,
        max_points
    )

    points = sorted(
        random.sample(
            range(1, total_steps),
            number_of_points
        )
```

```
        )

        points = [0] + points + [total_steps]

        child = []

        for index in range(len(points) - 1):
            start = points[index]
            end = points[index + 1]

            source = (
                parent1
                if index % 2 == 0
                else parent2
            )

            child.extend(
                source[start:end]
            )

        return clone_genes(child)


    def _crossover_drums(parent1, parent2):
        """
        Faz crossover independente para cada pista de bateria.

        Isso evita que Bat1, Bat2, BatI e BatF
        fiquem sempre geneticamente acoplados.
        """

        bat1 = _crossover_genes(
            parent1.bat1,
            parent2.bat1
        )

        bat2 = _crossover_genes(
```

```
            parent1.bat2,
            parent2.bat2
        )

        bati = _crossover_genes(
            parent1.bati,
            parent2.bati
        )

        batf = _crossover_genes(
            parent1.batf,
            parent2.batf
        )

        return bat1, bat2, bati, batf


# ============================================================
# CROSSOVER DE BASS
# ============================================================

def _mix_bass_pattern(parent1, parent2):
    """
    Mistura dois BassPatterns.

    O BassPattern precisa continuar preenchendo
    exatamente um compasso, portanto são testadas
    combinações válidas antes da criação do objeto.
    """

    if random.random() < 0.25:
        return BassPattern()

    candidates = [
        parent1,
        parent2,
    ]
```

```
    if random.random() < 0.5:
        source = random.choice(candidates)
        return clone_bass_pattern(source)

    count = random.choice(
        BassPattern.NOTE_COUNTS
    )

    source = random.choice(candidates)

    if len(source.notes) == count:
        notes = [
            type(note)(
                note.position,
                note.duration
            )
            for note in source.notes
        ]

        return BassPattern(notes)

    return BassPattern()


# ============================================================
# CROSSOVER DE ARP
# ============================================================

def _mix_arp_pattern(parent1, parent2):
    """
    Mistura note_count e step dos dois pais.

    Também existe uma chance de gerar um ArpPattern
    completamente novo.
    """
```

```
    if random.random() < 0.25:
        return ArpPattern()

    note_count = random.choice([
        parent1.note_count,
        parent2.note_count,
    ])

    step = random.choice([
        parent1.step,
        parent2.step,
    ])

    try:
        return ArpPattern(
            note_count=note_count,
            step=step
        )
    except ValueError:
        return ArpPattern()


# ============================================================
# CROSSOVER DE ACORDES
# ============================================================

def _chord_signature(chord):
    return (
        chord.root,
        chord.chord_type,
        getattr(chord, "inversion", 0),
    )


def _new_random_chord():
    """
    Cria um MChord novo, independente dos pais.
```

```
        """

        root = random.randint(
            MChord.ROOT_MIN_MIDI_NOTE,
            MChord.ROOT_MAX_MIDI_NOTE
        )

        chord_type = random.choice(
            tuple(
                MChord.CHORD_INTERVALS.keys()
            )
        )

        inversion = random.choice(
            tuple(MChord.INVERSIONS)
        )

        return MChord(
            root,
            chord_type,
            inversion
        )


    def _select_parent_chord(
        parent1,
        parent2,
        index,
        used
    ):
        """
        Seleciona um acorde com maior diversidade.

        Há três possibilidades:

        1. acorde da mesma posição;
        2. acorde de qualquer posição dos pais;
```

```
        3. novo acorde gerado aleatoriamente.
        """

        # Chance relativamente alta de criar
        # algo que não existia nos pais.
        if random.random() < 0.30:
            for _ in range(10):
                chord = _new_random_chord()

                if _chord_signature(chord) not in used:
                    return chord

        candidates = []

        # Acordes da mesma posição.
        candidates.extend([
            parent1.genotype.mchords[index],
            parent2.genotype.mchords[index],
        ])

        # Acordes de outras posições.
        if random.random() < 0.70:
            candidates.extend(
                chord
                for parent in (parent1, parent2)
                for chord in parent.genotype.mchords
            )

        # Remove duplicados.
        unique = {}

        for chord in candidates:
            unique[
                _chord_signature(chord)
            ] = chord

        candidates = list(
```

```
            unique.values()
        )

        unused = [
            chord
            for chord in candidates
            if _chord_signature(chord)
            not in used
        ]

        if unused:
            return random.choice(
                unused
            )

        return random.choice(
            candidates
        )


# ============================================================
# CROSSOVER PRINCIPAL
# ============================================================

def crossover(parent1, parent2):
    """
    Crossover agressivo entre dois indivíduos.

    O objetivo é produzir filhos com maior diversidade
    genética, mantendo a validade estrutural do Genotype.
    """

    # Quando o crossover não acontece, ainda existe
    # uma pequena chance de usar o segundo pai.
    if random.random() > CROSSOVER_RATE:

        if random.random() < 0.20:
```

```
            return clone_individual(parent2)

        return clone_individual(parent1)

    # ------------------------------------------------------
    # BATERIA
    # ------------------------------------------------------

    bat1, bat2, bati, batf = _crossover_drums(
        parent1,
        parent2
    )

    # ------------------------------------------------------
    # BASS
    # ------------------------------------------------------

    bass_patt1 = _mix_bass_pattern(
        parent1.bass_patt1,
        parent2.bass_patt1
    )

    bass_patt2 = _mix_bass_pattern(
        parent1.bass_patt2,
        parent2.bass_patt2
    )

    # ------------------------------------------------------
    # ARP
    # ------------------------------------------------------

    arp_patt1 = _mix_arp_pattern(
        parent1.arp_patt1,
        parent2.arp_patt1
    )

    arp_patt2 = _mix_arp_pattern(
```

```
            parent1.arp_patt2,
            parent2.arp_patt2
        )

        # ------------------------------------------------------
        # MCHORDS
        # ------------------------------------------------------

        chords = []
        used = set()

        for index in range(NUM_MCHORDS):

            chord = _select_parent_chord(
                parent1,
                parent2,
                index,
                used
            )

            chord = clone_chord(
                chord
            )

            chords.append(
                chord
            )

            used.add(
                _chord_signature(chord)
            )

        # ------------------------------------------------------
        # FILHO
        # ------------------------------------------------------

        return make_child(
```

```
            bat1,
            bat2=bat2,
            bati=bati,
            batf=batf,
            bass_patt1=bass_patt1,
            bass_patt2=bass_patt2,
            arp_patt1=arp_patt1,
            arp_patt2=arp_patt2,
            mchords=chords,
        )
```

O que mudou de forma importante

A diferença principal é que agora o crossover **não é apenas uma recombinação conservadora dos pais**.

Por exemplo, na bateria:

```
Pai 1: A A A A | A A A A | A A A A | A A A A
Pai 2: B B B B | B B B B | B B B B | B B B B

Filho: A A B B | B A A B | A A A A | B B A A
```

E isso acontece **independentemente em Bat1, Bat2, BatI e BatF**.

Para os acordes, existe ainda uma chance de criar um acorde que **nenhum dos dois pais possui**:

```
Pai 1 → C major, F major, G major...
Pai 2 → A minor, D minor, E major...

Filho → C major, D minor, G# minor, F major...
                          ↑
                    novo acorde
```

Isso é particularmente importante no seu caso, porque os `MChord`s influenciam bastante o espaço musical disponível para **Bass** e **Arp**.

Uma observação: eu deixei o crossover de `BassPattern` relativamente conservador porque ele possui uma restrição estrutural forte — as durações precisam preencher exatamente um compasso. Tentar simplesmente cruzar notas de dois `BassPattern` s pode produzir padrões inválidos. **A mutation é o lugar mais adequado para tornar Bass/Arp realmente agressivos.**

Então eu faria exatamente como você propôs: **primeiro testar este `crossover.py` ; depois me passe o `mutation.py`** , e aí podemos tornar a mutação complementar a esse crossover, principalmente para criar diversidade que **não dependa dos pais**.

Faça o procedimento análogo no mutation.py:

```
import random



from constants import \*

from genotype import ArpPattern, BassPattern, Gene, MChord




def generate_drum_step(position, profile="base"):

&#x20;  """

&#x20;  Gera um Gene de bateria para determinado step.
…
```

Show more

Sim. Aqui eu faria a mutação **complementar ao crossover agressivo** que acabamos de criar.

A principal mudança será: **quando um componente é escolhido para mutação, ele terá uma chance muito maior de realmente mudar**, em vez de frequentemente produzir algo muito parecido com o original.

Também vou manter a interface atual e todos os nomes externos (`mutate` , `mutate_track` , `mutate_mchords` , etc.), para não exigir alterações no restante do projeto.

mutation.py — versão mais agressiva

```
import random

from constants import *

from genotype import (
    ArpPattern,
    BassPattern,
    Gene,
    MChord,
)


# ============================================================
# GERAÇÃO DE UM STEP DE BATERIA
# ============================================================

def generate_drum_step(position, profile="base"):
    """
    Gera um Gene de bateria para determinado step.
    """

    profile_values = {
        "base": (
            BAT1_EMPTY_STEP_PROBABILITY,
            BAT1_SECOND_NOTE_PROBABILITY,
            BAT1_THIRD_NOTE_PROBABILITY,
        ),
        "chorus": (
            BAT2_EMPTY_STEP_PROBABILITY,
            BAT2_SECOND_NOTE_PROBABILITY,
            BAT2_THIRD_NOTE_PROBABILITY,
        ),
        "intro": (
            BATI_EMPTY_STEP_PROBABILITY,
            BATI_SECOND_NOTE_PROBABILITY,
```

```
            BATI_THIRD_NOTE_PROBABILITY,
        ),
        "final": (
            BATF_EMPTY_STEP_PROBABILITY,
            BATF_SECOND_NOTE_PROBABILITY,
            BATF_THIRD_NOTE_PROBABILITY,
        ),
    }

    (
        empty_probability,
        second_note_probability,
        third_note_probability,
    ) = profile_values[profile]

    if random.random() < empty_probability:
        return Gene()

    candidates = []
    weights = []

    candidates.append(KICK)

    if position in (0, 4):
        weights.append(KICK_WEIGHT)
    elif position in (2, 6):
        weights.append(22)
    else:
        weights.append(8)

    candidates.append(SNARE)

    if position in (2, 6):
        weights.append(SNARE_WEIGHT)
    elif position in (1, 3, 5, 7):
        weights.append(5)
    else:
```

```
            weights.append(2)

        candidates.append(CLOSED_HIHAT)
        weights.append(CLOSED_HIHAT_WEIGHT)

        candidates.append(OPEN_HIHAT)

        if position in (3, 7):
            weights.append(OPEN_HIHAT_WEIGHT * 2)
        else:
            weights.append(OPEN_HIHAT_WEIGHT)

        candidates.append(CRASH)

        if position == 0:
            weights.append(CRASH_WEIGHT * 3)
        else:
            weights.append(1)

        first = random.choices(
            candidates,
            weights=weights,
            k=1
        )[0]

        notes = [first]

        if (
            Gene.MAX_NOTES >= 2
            and random.random() < second_note_probability
        ):
            available = [
                drum
                for drum in candidates
                if drum not in notes
            ]
```

```
            if available:
                available_weights = [
                    weights[candidates.index(drum)]
                    for drum in available
                ]

                second = random.choices(
                    available,
                    weights=available_weights,
                    k=1
                )[0]

                if not (
                    (
                        second == CLOSED_HIHAT
                        and OPEN_HIHAT in notes
                    )
                    or (
                        second == OPEN_HIHAT
                        and CLOSED_HIHAT in notes
                    )
                ):
                    notes.append(second)

        if (
            Gene.MAX_NOTES >= 3
            and random.random() < third_note_probability
        ):
            available = [
                drum
                for drum in candidates
                if drum not in notes
            ]

            if available:
                third = random.choice(available)
```

```
            if not (
                (
                    third == CLOSED_HIHAT
                    and OPEN_HIHAT in notes
                )
                or (
                    third == OPEN_HIHAT
                    and CLOSED_HIHAT in notes
                )
            ):
                notes.append(third)

    notes.sort()

    return Gene(notes)


# ============================================================
# MUTAÇÃO DE UM STEP DE BATERIA
# ============================================================

def mutate_drum_step(track, index, profile="base"):
    """
    Muta um Gene de bateria.

    A mutação é deliberadamente agressiva:
    - remove elementos;
    - adiciona elementos;
    - troca elementos;
    - ou recria completamente o step.
    """

    old_notes = list(track[index].notes)

    action = random.random()

    # ------------------------------------------------------
```

```
    # Gene vazio
    # ------------------------------------------------------

    if not old_notes:

        # Em um Gene vazio, normalmente criamos algo novo.
        track[index] = generate_drum_step(
            index,
            profile=profile
        )

        return

    # ------------------------------------------------------
    # Recriação completa
    # ------------------------------------------------------

    if action < 0.35:

        new_gene = generate_drum_step(
            index,
            profile=profile
        )

        # Tenta evitar que a mutação não produza mudança.
        for _ in range(5):

            if new_gene.notes != old_notes:
                track[index] = new_gene
                return

            new_gene = generate_drum_step(
                index,
                profile=profile
            )

        track[index] = new_gene
```

```
            return

        # ------------------------------------------------------------
        # Remoção
        # ------------------------------------------------------------

        if action < 0.50:

            if old_notes:

                position = random.randrange(
                    len(old_notes)
                )

                old_notes.pop(position)

            track[index] = Gene(
                sorted(set(old_notes))
            )

            return

        # ------------------------------------------------------------
        # Adição
        # ------------------------------------------------------------

        if action < 0.68:

            available = [
                drum
                for drum in DRUMS
                if drum not in old_notes
            ]

            if (
                available
                and len(old_notes) < Gene.MAX_NOTES
```

```
            ):

                random.shuffle(
                    available
                )

                for new_drum in available:

                    if (
                        new_drum == CLOSED_HIHAT
                        and OPEN_HIHAT in old_notes
                    ):
                        continue

                    if (
                        new_drum == OPEN_HIHAT
                        and CLOSED_HIHAT in old_notes
                    ):
                        continue

                    old_notes.append(
                        new_drum
                    )

                    break

            track[index] = Gene(
                sorted(set(old_notes))
            )

            return

        # ----------------------------------------------------------
        # Substituição
        # ----------------------------------------------------------

        if action < 0.88:
```

```
        if old_notes:

            position = random.randrange(
                len(old_notes)
            )

            current = old_notes[position]

            available = [
                drum
                for drum in DRUMS
                if drum != current
                and drum not in old_notes
            ]

            random.shuffle(
                available
            )

            for new_drum in available:

                other_notes = [
                    note
                    for i, note in enumerate(old_notes)
                    if i != position
                ]

                if (
                    new_drum == CLOSED_HIHAT
                    and OPEN_HIHAT in other_notes
                ):
                    continue

                if (
                    new_drum == OPEN_HIHAT
                    and CLOSED_HIHAT in other_notes
```

```
                ):
                    continue

                old_notes[position] = new_drum
                break

        track[index] = Gene(
            sorted(set(old_notes))
        )

        return

    # ------------------------------------------------------
    # Mutação extrema
    # ------------------------------------------------------

    track[index] = Gene()

    if random.random() < 0.75:
        track[index] = generate_drum_step(
            index,
            profile=profile
        )


# ============================================================
# MUTAÇÃO DE MCHORD
# ============================================================

def mutate_mchord(chord):
    """
    Gera outro MChord válido.

    A mutação é agressiva e pode alterar:
    - raiz;
    - tipo;
    - inversão;
```

```
    - raiz + tipo;
    - todos os componentes.
    """

    chord_types = tuple(
        MCHORD_INTERVALS
    )

    inversions = tuple(
        MCHORD_INVERSIONS
    )

    original = (
        chord.root,
        chord.chord_type,
        chord.inversion,
    )

    # ------------------------------------------------------
    # Tentativas de mutação
    # ------------------------------------------------------

    for _ in range(100):

        action = random.random()

        # ------------------------------------------------
        # Alteração principalmente harmônica
        # ------------------------------------------------

        if action < 0.30:

            root = random.randint(
                MCHORD_ROOT_MIN,
                MCHORD_ROOT_MAX
            )
```

```
            chord_type = random.choice(
                chord_types
            )

            inversion = chord.inversion

        # ------------------------------------------------
        # Alteração de tipo + inversão
        # ------------------------------------------------

        elif action < 0.50:

            root = chord.root

            chord_type = random.choice(
                chord_types
            )

            inversion = random.choice(
                inversions
            )

        # ------------------------------------------------
        # Alteração da raiz
        # ------------------------------------------------

        elif action < 0.70:

            root = random.randint(
                MCHORD_ROOT_MIN,
                MCHORD_ROOT_MAX
            )

            chord_type = chord.chord_type

            inversion = random.choice(
                inversions
```

```
            )

        # ------------------------------------------------------
        # Mutação completamente aleatória
        # ------------------------------------------------------

        else:

            root = random.randint(
                MCHORD_ROOT_MIN,
                MCHORD_ROOT_MAX
            )

            chord_type = random.choice(
                chord_types
            )

            inversion = random.choice(
                inversions
            )

        candidate = MChord(
            root,
            chord_type,
            inversion
        )

        candidate_signature = (
            candidate.root,
            candidate.chord_type,
            candidate.inversion,
        )

        if candidate_signature != original:
            return candidate

    # ------------------------------------------------------
```

```
        # Fallback
        # ----------------------------------------------------

        alternatives = [
            inversion
            for inversion in inversions
            if inversion != chord.inversion
        ]

        if alternatives:

            return MChord(
                chord.root,
                chord.chord_type,
                random.choice(alternatives)
            )

        return MChord(
            chord.root,
            chord.chord_type,
            chord.inversion
        )


# ============================================================
# RANDOMIZAÇÃO DE INVERSÕES
# ============================================================

def randomize_chord_inversions(individual):
    """
    Atribui uma inversão aleatória a cada MChord.
    """

    for index, chord in enumerate(
        individual.genotype.mchords,
        start=1
    ):
```

```
        current_inversion = chord.inversion

        available = [
            inversion
            for inversion in MCHORD_INVERSIONS
            if inversion != current_inversion
        ]

        if available:
            inversion = random.choice(
                available
            )
        else:
            inversion = current_inversion

        setattr(
            individual.genotype,
            f"mchord{index}",
            MChord(
                chord.root,
                chord.chord_type,
                inversion,
            ),
        )


# ============================================================
# MUTAÇÃO DE TRACK DE BATERIA
# ============================================================

def mutate_track(
    track,
    mutation_rate,
    profile="base"
):
    """
```

```
    Aplica mutação independente a cada step.

    A probabilidade recebida em mutation_rate continua
    controlando quantos steps serão afetados.
    """

    for index in range(len(track)):

        if random.random() < mutation_rate:

            mutate_drum_step(
                track,
                index,
                profile=profile
            )


# ============================================================
# MUTAÇÃO DOS MCHORDS
# ============================================================

def mutate_mchords(individual):
    """
    Muta cada MChord independentemente.

    Vários acordes podem ser modificados na mesma rodada.
    """

    for index, chord in enumerate(
        individual.genotype.mchords,
        start=1
    ):

        if random.random() < MCHORD_MUTATION_RATE:

            setattr(
                individual.genotype,
```

```
                f"mchord{index}",
                mutate_mchord(chord)
            )


# ============================================================
# MUTAÇÃO DE BASS
# ============================================================

def mutate_bass_pattern(pattern):
    """
    Gera uma nova estrutura rítmica de baixo.

    A nova estrutura é independente da anterior.
    """

    # Na maioria das vezes, gera completamente
    # uma nova estrutura.
    if random.random() < 0.85:
        return BassPattern()

    # Pequena chance de simplesmente manter
    # a estrutura atual.
    return BassPattern(
        [
            type(note)(
                note.position,
                note.duration
            )
            for note in pattern.notes
        ]
    )


# ============================================================
# MUTAÇÃO DE ARP
# ============================================================
```

```
def mutate_arp_pattern(pattern):
    """
    Gera uma nova estrutura rítmica de arpejo.
    """

    if random.random() < 0.85:
        return ArpPattern()

    return ArpPattern(
        note_count=pattern.note_count,
        step=pattern.step
    )


# ============================================================
# MUTAÇÃO PRINCIPAL
# ============================================================

def mutate(individual):
    """
    Aplica mutação independente a cada componente
    do indivíduo.

    A mutação é propositalmente agressiva para evitar
    convergência prematura da população.
    """

    # ------------------------------------------------------
    # BATERIA
    # ------------------------------------------------------

    mutate_track(
        individual.bat1,
        MUTATION_RATE
    )
```

```
    mutate_track(
        individual.bat2,
        BAT2_MUTATION_RATE,
        profile="chorus"
    )

    mutate_track(
        individual.bati,
        BATI_MUTATION_RATE,
        profile="intro"
    )

    mutate_track(
        individual.batf,
        BATF_MUTATION_RATE,
        profile="final"
    )

    # ---------------------------------------------------------
    # MCHORDS
    # ---------------------------------------------------------

    mutate_mchords(
        individual
    )

    # ---------------------------------------------------------
    # BASS PATTERN 1
    # ---------------------------------------------------------

    if random.random() < BASS_PATT1_MUTATION_RATE:

        individual.bass_patt1 = (
            mutate_bass_pattern(
                individual.bass_patt1
            )
        )
```

```
    # ------------------------------------------------------
    # BASS PATTERN 2
    # ------------------------------------------------------

    if random.random() < BASS_PATT2_MUTATION_RATE:

        individual.bass_patt2 = (
            mutate_bass_pattern(
                individual.bass_patt2
            )
        )

    # ------------------------------------------------------
    # ARP PATTERN 1
    # ------------------------------------------------------

    if random.random() < ARP_PATT1_MUTATION_RATE:

        individual.arp_patt1 = (
            mutate_arp_pattern(
                individual.arp_patt1
            )
        )

    # ------------------------------------------------------
    # ARP PATTERN 2
    # ------------------------------------------------------

    if random.random() < ARP_PATT2_MUTATION_RATE:

        individual.arp_patt2 = (
            mutate_arp_pattern(
                individual.arp_patt2
            )
        )
```

```
    return individual
```

O ponto mais importante

Agora temos duas fontes de diversidade trabalhando juntas:

Crossover:

- mistura material dos dois pais;
- pode criar novos MChords;
- mistura segmentos diferentes das baterias;
- pode gerar novos Bass/Arp patterns.

Mutation:

- pode criar material que **não existe em nenhum dos pais**;
- bateria pode ser recriada completamente;
- MChords podem mudar raiz + tipo + inversão;
- Bass/Arp podem ganhar uma estrutura nova.

Isso deve reduzir bastante a tendência da população de ficar parecida.

Um detalhe importante: **eu não aumentei artificialmente `MUTATION_RATE`, `BASS_PATT1_MUTATION_RATE`, etc. dentro do código**. Esses valores continuam sendo controlados pelo seu `constants.py`. Ou seja, primeiro podemos testar exatamente com as taxas atuais e observar o quanto a diversidade aumentou.

Se mesmo assim os indivíduos continuarem muito semelhantes, aí o próximo arquivo que eu analisaria seria o **`constants.py`**, porque pode acontecer de o crossover/mutation estar agressivo, mas as probabilidades em `constants.py` serem tão baixas que quase nada seja efetivamente alterado.

> Novamente não notei variabilidades kkk, e os códigos ficaram maiores, mas tudo bem vou deixar assim

kkkk 😄 Sim, isso é um sinal importante: **provavelmente o problema não está mais no `genotype.py`, `crossover.py` ou `mutation.py` isoladamente**.

E concordo em deixar esses códigos como estão por enquanto. Não vale a pena continuar aumentando a complexidade sem saber onde a diversidade está sendo perdida.

O próximo passo que eu faria é **investigar**, não modificar.

Principalmente nesta ordem:

1. **`constants.py`**
   Verificar as taxas reais de `MUTATION_RATE`, `BASS_PATT*_MUTATION_RATE`, `MCHORD_MUTATION_RATE`, `CROSSOVER_RATE` etc.
2. **`individual.py`**
   Verificar se os indivíduos estão realmente armazenando os novos genes/padrões ou se alguma coisa está sendo reconstruída/copied de forma que elimine a diversidade.
3. **`genetic_algorithm.py`**
   Esse é provavelmente o mais importante agora:
   - seleção;
   - elitismo;
   - número de indivíduos;
   - quantos filhos são realmente criados;
   - quando `crossover()` é chamado;
   - quando `mutate()` é chamado;
   - se os filhos estão sendo substituídos por clones;
   - se a população está convergindo rapidamente.
4. **Medir diversidade diretamente**
   Em vez de olhar apenas para as músicas, podemos imprimir algo como:

```
Plain text

Geração 1
  MChords diferentes: 87/100
  BassPatt diferentes: 64/100
  ArpPatt diferentes: 52/100
  Bat1 diferentes: 91/100

Geração 10
  MChords diferentes: 23/100
  BassPatt diferentes: 8/100
  ArpPatt diferentes: 11/100
  Bat1 diferentes: 31/100
```

Aí vamos descobrir **exatamente em qual etapa a diversidade desaparece**.

Na verdade, pelo que você descreveu — *"os resultados continuam muito parecidos apesar de termos tornado inicialização, crossover e mutation mais aleatórios"* — eu suspeitaria bastante de **seleção/elitismo ou de alguma lógica em `genetic_algorithm.py`**.

Então eu pararia de mexer nesses três arquivos por enquanto. 😄

Se você me passar o **`constants.py`**, podemos primeiro conferir se as taxas estão razoáveis. Depois eu olharia o `individual.py` e o `genetic_algorithm.py`.

Pasted markdown…
Code

Aqui está o código evolve_algorithm.py, primeiro me diga resumidamente o que da pra melhorar pra aumentar a diversidade

Sim. Olhando especificamente para o `evolve_algorithm.py`, há **alguns pontos bem mais importantes** do que continuar deixando `crossover.py` e `mutation.py` mais agressivos.

1. O maior suspeito: seleção por torneio

Você está usando torneio de tamanho **3**:

```python
candidates = random.sample(
    range(len(population)),
    tournament_size
)
```

e sempre escolhe o melhor dos três. Pasted markdown

Isso gera uma **pressão seletiva considerável**. Se alguns indivíduos obtêm fitness um pouco maior, eles começam a ser escolhidos como pais com muita frequência.

Para aumentar diversidade, eu consideraria:

- torneio de 2 em vez de 3;
- ou seleção probabilística;
- ou manter torneio 3, mas adicionar mecanismos para evitar repetir os mesmos pais.

Esse é um candidato forte.

---

2. O elitismo mantém um clone exato

Você mantém o melhor indivíduo:

Python

```
if ELITISM:
    elite = population[best_index]
    new_population.append(
        clone_individual(elite)
    )
```

Pasted markdown

Isso sozinho não é um grande problema porque é apenas **1 indivíduo**, mas combinado com seleção forte pode acelerar a convergência.

Eu **não removeria o elitismo imediatamente**. Primeiro reduziria a pressão de seleção.

---

3. O `RANDOM_IMMIGRANT_RATE` é uma ótima ideia — mas depende do valor

Você já possui exatamente o mecanismo que eu colocaria para combater convergência:

Python

```
if random.random() < RANDOM_IMMIGRANT_RATE:
    new_population.append(
        initialize_individual()
    )
```

Pasted markdown

Então **não precisamos inventar outro mecanismo**.

A questão é: qual é o valor de `RANDOM_IMMIGRANT_RATE` no `constants.py` ?

Se for algo como:

Python

```
0.01
```

em uma população de 100 indivíduos, você terá em média **1 imigrante por geração**.

Isso pode ser pouco.

Algo como `0.10` produziria aproximadamente **10 indivíduos completamente novos por geração**.

Mas eu não mudaria isso ainda sem ver os `constants.py`.

---

4. Há uma coisa interessante na inicialização

Você está explicitamente gerando cada step das quatro baterias:

Python

```
individual.bat1[step] = generate_drum_step(step)
individual.bat2[step] = generate_drum_step(step, profile="chorus")
individual.bati[step] = generate_drum_step(step, profile="intro")
individual.batf[step] = generate_drum_step(step, profile="final")
```

Pasted markdown

Isso é bom.

Porém, depois você faz:

Python

```
randomize_chord_inversions(individual)
```

Pasted markdown

Ou seja, pelo comentário do próprio código, **a inicialização está preservando raiz/tipo/ordem dos acordes e variando apenas inversões**.

Isso pode ser uma fonte importante de similaridade harmônica **se os MChords criados em `Individual()` / `Genotype()` ainda estiverem muito parecidos**.

---

5. O ciclo reprodutivo em si está correto

Esta parte está conceitualmente boa:

Plain text

```
seleção
   ↓
parent1 + parent2
   ↓
crossover
   ↓
mutation
   ↓
novo indivíduo
```

O código realmente faz isso:

```python
parent1 = selection(...)
parent2 = selection(...)

child = crossover(parent1, parent2)
child = mutate(child)

new_population.append(child)
```

Pasted markdown

Portanto, **não vejo aqui um erro óbvio do tipo "a mutation nunca é chamada"**.

Minha suspeita atual

Eu colocaria assim:

Mais provável → menos provável:

1. 🟥 **`constants.py` está usando taxas muito conservadoras**
2. 🟥 **Seleção por torneio 3 está fazendo a população convergir**
3. 🟧 `Individual()` /estrutura interna pode estar limitando a diversidade
4. 🟨 Elitismo + seleção estão reforçando essa convergência
5. 🟩 `crossover.py` / `mutation.py` — agora já estão razoavelmente agressivos

E tem uma coisa que eu faria **antes de mudar qualquer outro algoritmo**:

> **Medir a diversidade da população a cada geração.**

Porque atualmente seu log mostra:

Plain text

```
Mínima
Média
Melhor
Global
```

Pasted markdown

mas não mostra **quantos indivíduos são geneticamente diferentes**.

Por exemplo:

Plain text

```
Geração 1:
  indivíduos únicos: 100/100

Geração 10:
  indivíduos únicos: 74/100

Geração 20:
  indivíduos únicos: 18/100

Geração 30:
  indivíduos únicos: 7/100
```

Se aparecer algo assim, teremos a prova de que **a população está convergindo**.

Se, por outro lado, aparecer:

Plain text

```
Geração 30:
100/100 indivíduos diferentes
```

e as músicas ainda soarem parecidas, então o problema provavelmente está **na representação genética ou na geração MIDI**, e não no GA.

Próximo passo que eu recomendo: me passe o `constants.py`. Aí conseguimos ver as taxas reais antes de mexer novamente no algoritmo.

O que da pra melhorar na seleção?

Sim. Olhando especificamente para a **seleção** do seu `evolve_algorithm.py`, eu vejo algumas melhorias importantes para aumentar a diversidade sem simplesmente deixar a mutação cada vez mais agressiva.

Atualmente você usa **torneio de tamanho 3**: escolhe 3 indivíduos aleatoriamente e sempre pega o melhor deles. Pasted markdown

1. Reduzir o torneio de 3 para 2

Essa seria a primeira mudança que eu testaria.

Hoje:

```python
tournament_size = min(3, len(population))
```

Com 3 indivíduos, a pressão seletiva é relativamente alta:

```
3 candidatos → sempre escolhe o melhor
```

Com 2:

```
2 candidatos → escolhe o melhor
```

Isso permite que indivíduos medianos tenham mais chance de continuar na população.

Efeito esperado:

```
Torneio 3
   ↓
convergência rápida
   ↓
indivíduos ficam parecidos
```

versus:

```
Torneio 2
   ↓
mais diversidade
   ↓
convergência mais lenta
   ↓
mais exploração
```

Eu começaria por aqui.

---

2. Não usar somente o fitness para decidir quem reproduz

Esse é provavelmente o próximo passo mais interessante.

Atualmente:

```
fitness
   ↓
seleção
   ↓
crossover
   ↓
mutação
```

O problema é que dois indivíduos podem ter:

```
Fitness A = 0.850
Fitness B = 0.848
```

mas serem **geneticamente muito diferentes**.

O algoritmo considera A melhor e tende a reproduzi-lo mais.

Podemos fazer:

```
fitness + diversidade
       ↓
seleção
```

Por exemplo:

```
score_selecao =
    0.8 * fitness +
    0.2 * diversidade
```

Assim, um indivíduo ligeiramente pior no fitness, mas bastante diferente dos demais, ainda pode ser selecionado.

Isso combina muito bem com seu problema musical.

---

3. Penalizar indivíduos duplicados ou quase duplicados

Outra possibilidade é impedir que a população fique assim:

```
A
A
A
A'
A'
B
B
B
```

Mesmo que o fitness esteja bom.

Podemos calcular uma **assinatura genética** do indivíduo:

```
Bat1
Bat2
```

```
BatI
BatF
BassPattern1
BassPattern2
ArpPattern1
ArpPattern2
MChord1...MChord8
```

e detectar:

Plain text

```
indivíduo 1 == indivíduo 7
```

ou até similaridade:

Plain text

```
95% dos genes iguais
```

Nesse caso, a seleção poderia dar preferência ao indivíduo diferente.

---

4. Cuidado com o elitismo

Seu algoritmo preserva diretamente o melhor indivíduo da geração. Pasted markdown

Isso é bom para não perder o melhor resultado, mas combinado com:

Plain text

```
Elitismo
+
torneio 3
+
crossover
+
mutação
```

pode acelerar bastante a convergência.

Eu **não removeria o elitismo** inicialmente.

Em vez disso, manteria:

```python
ELITISM = True
```

mas apenas **1 indivíduo elitista**.

Se atualmente já é 1, está ótimo.

---

5. Manter os imigrantes aleatórios

Essa parte do seu algoritmo é uma boa ideia:

```
população
   ↓
alguns indivíduos totalmente novos
```

Seu código já possui `RANDOM_IMMIGRANT_RATE`. Pasted markdown

Eu manteria isso.

Na prática, os imigrantes funcionam como uma espécie de:

> "injeção de material genético novo"

e podem ser particularmente úteis no seu caso porque você está observando **BassPattern e ArpPattern muito semelhantes**.

O que eu faria primeiro

Eu **não mexeria em tudo de uma vez**.

Faria nesta ordem:

Experimento 1

Mudar:

Python

```
tournament_size = 3
```

para:

Python

```
tournament_size = 2
```

e observar.

Experimento 2

Se ainda houver pouca diversidade:

Plain text

```
torneio 2
+
imigrantes aleatórios
+
elitismo 1
```

e medir a diversidade real da população.

Experimento 3

Se ainda convergir muito:

Plain text

```
fitness
+
diversidade genética
```

na seleção.

Minha principal recomendação: antes de fazer uma seleção muito sofisticada, eu adicionaria ao `evolve_algorithm.py` um relatório por geração como:

Plain text

```
Generation 10
Best fitness: 0.872
Mean fitness: 0.811

Unique individuals: 37/100
Unique BassPattern1: 12/100
Unique ArpPattern1: 9/100
Unique MChords: 31/100
Unique Bat1: 18/100
```

Aí conseguimos descobrir **exatamente onde a diversidade está morrendo**. Isso é melhor do que ficar aumentando aleatoriamente crossover/mutation.

qual variável tenho que mudar?

Para reduzir a pressão seletiva, a variável que você deve mudar é o **tamanho do torneio** dentro da função de seleção.

No seu `evolve_algorithm.py`, atualmente está:

Python

```
tournament_size = min(3, len(population))
```

Mude para:

Python

```
tournament_size = min(2, len(population))
```

Ou seja:

3 → 2

Essa é a alteração que eu faria primeiro para aumentar a diversidade. Pasted markdown

Não precisa alterar `CROSSOVER_RATE` ou `MUTATION_RATE` ainda.

Olhe o código a seguir e ajuste-o para fazer o seguinte:

Na função diversify_midi:

Se diversify_midi_notes == True ele continua fazendo a diversificação de notas que ele já está fazendo, se não ele não faz.

Acrescente também o seguinte:

Se diversify_midi_instruments == True ele vai executar uma função de diversificação dos instrumentos da música. Para essa segunda parte, considere os seguintes grupos de instrumentos com base nos nomes das pistas:

Drums: Diversificar o tipo do canal de bateria usado, pois existem vários no mapa midi.
String1: Diversificar o tipo de String usado considerando apenas instrumentos de String.
String2: Diversificar o tipo de String usado considerando
…

Show more

Sim. Eu faria a alteração mantendo a diversificação de notas completamente independente da diversificação de instrumentos.

A lógica ficará:

- `diversify_midi_notes=True` → executa exatamente a diversificação de notas atual.
- `diversify_midi_notes=False` → **não adiciona nenhuma nota**.
- `diversify_midi_instruments=True` → procura as pistas pelo nome e troca aleatoriamente o instrumento/programa de acordo com o grupo.
- `diversify_midi_instruments=False` → não altera os instrumentos.
- A pista `Harmony` pode receber qualquer instrumento melódico razoável.
- O nome da pista (`instrument.name`) é preservado.
- A troca de instrumento é feita pelo **program/program number do General MIDI**.
- Para `Drums` , mantemos `is_drum=True` e diversificamos o `program` associado ao canal de bateria.
- Também corrigi a chamada da CLI, que no código original estava passando `args.seed` como terceiro argumento posicional, fazendo-o cair em `diversify_notes` .

Como o seu código não trouxe um mapa externo de instrumentos, usei os nomes General MIDI fornecidos pelo próprio `pretty_midi` .

Python

```
import argparse
import random

import pretty_midi


# ============================================================
# CONFIGURAÇÃO
# ============================================================

DEFAULT_BPM = 120.0


# A música é dividida em 8 partes.
# Cada valor representa a probabilidade de inserir
# uma nota naquela parte.

PART_PROBABILITIES = [
    0.00,  # Parte 1
    0.00,  # Parte 2
    0.10,  # Parte 3
```

Como fica o comportamento

Por exemplo, somente notas:

Bash

```
python diversify_midi.py input.mid output.mid --diversify-notes
```

Somente instrumentos:

Bash

```
python diversify_midi.py input.mid output.mid --diversify-instruments
```

Ambos:

Bash

```
python diversify_midi.py input.mid output.mid \
    --diversify-notes \
    --diversify-instruments
```

E com seed:

```bash
python diversify_midi.py input.mid output.mid \
    --diversify-instruments \
    --seed 123
```

Um ponto importante: **não alterei o conteúdo das notas quando `diversify_midi_notes=False`**. Nesse caso, a única alteração possível é o `program` do instrumento.

Também preservei `instrument.name`. Portanto, se a pista originalmente é:

```
String1
```

ela continuará sendo chamada `String1`, mesmo que seu instrumento passe, por exemplo, de `Violin` para `Cello`.

Observação sobre `Drums`: no General MIDI, bateria é uma situação especial: normalmente o canal 10 define a percussão e os diferentes sons são definidos pelas notas/pitches (36 Kick, 38 Snare etc.), e não por trocar o `program`. Por isso, nessa implementação a parte de instrumentos de `Drums` altera o `program` associado, mas mantém `is_drum=True`. Se o seu "mapa MIDI" tiver uma estrutura específica de **diferentes mapas de bateria**, aí vale adaptar `DRUM_PROGRAMS` para esse mapa em vez de usar os programas GM.